

QUALITY ASSURANCE PLAN

INDICE A / 05 MAI 2025

INTRODUCTION

Contexte

Ce document présente le plan d'assurance qualité pour le développement du jeu mobile « Cashou ». Il définit les stratégies, processus et responsabilités de l'équipe afin de garantir que le produit final respecte les exigences fonctionnelles, de performance, d'ergonomie et de sécurité.

Objectifs du document

- Décrire la méthodologie de test et les critères d'acceptation
- Définir les rôles et responsabilités de l'équipe de développement et d'assurance qualité
- Identifier les environnements de test et les outils utilisés
- Planifier et suivre l'ensemble des activités QA tout au long du projet

PORTÉE

Ce qui sera testé

- Toutes les fonctionnalités du jeu solo (campagne, niveaux, progression, scoring).
- Toutes les fonctionnalités du jeu multijoueur (création/rejoindre partie, déroulement du jeu synchrone/asynchrone, classement).
- Le système de quiz quotidiens (affichage, interaction, correction, feedback).
- L'exactitude et la clarté du contenu pédagogique intégré.
- L'interface utilisateur et l'expérience utilisateur sur les plateformes cibles définies.
- La gestion des comptes utilisateurs (si applicable).
- La persistance des données (sauvegarde de progression, scores).
- La compatibilité sur les navigateurs/OS/appareils listés en annexe.
- Les aspects de sécurité de base (ex: pas d'injection SQL simple, protection des données si comptes utilisateurs).
- La performance de base (temps de réponse sous charge normale).

Ce qui ne sera pas testé

- Tests de charge exhaustifs et stress tests poussés.
- Tests de sécurité avancés (pentesting) – sauf si planifié spécifiquement.
- Validation de l'infrastructure serveur sous-jacente (gérée par l'équipe Ops/Infra).
- Tests sur des configurations (matériel, OS, navigateurs) non explicitement listées comme cibles.
- Audit approfondi du code source (sauf revues de code ponctuelles).

|GESTION ET SUIVI DES ANOMALIES

Afin de répondre aux différentes anomalies et incidents pouvant intervenir pendant le développement du projet, la mise en place d'un outil de gestion et de suivi des tickets apparaît comme une évidence pour répondre aux besoins de support et de correction de façon efficace.

Outil

Afin de conserver un accès centralisé à nos différentes ressources, Notion a été choisi pour faire la gestion et le suivi des anomalies.

Un onglet "Suivi Anomalies" a été créé comprenant :

- Un tableau Kanban Global
- Un tableau Kanban "Mes tickets en cours"
- Un tableau classique "Historique"
- Une page base de connaissances

Cela va permettre à tous les collaborateurs d'avoir une vue générale et une possibilité d'action sur tous les tickets tout en leur permettant d'avoir un espace à eux, affinant cette vue et facilitant la lisibilité. L'historique sera lui conservé en cas de besoin consultation future.

En complément, une page "Base de connaissances" a été ajoutée. Cela va permettre de créer de la documentation sur la résolution de certains incidents. Cette base servira donc de marche à suivre en se basant sur des problèmes déjà rencontrés et permettra donc d'accélérer le traitement de certains tickets.

Contenu d'un ticket

Tout ticket à la même composition ayant pour but de donner des informations essentielles à son traitement. Cela va permettre d'effectuer une résolution efficace du ticket et aussi de pouvoir suivre en détail son avancement.

Un ticket contient obligatoirement (les items en **rouge** doivent être obligatoirement renseignés à la création, ceux en **bleu** le sont automatiquement, ceux en **noir** sont non obligatoire ou à remplir lors d'une prochaine étape) :

- Un ID
- Un titre
- Une assignation
- Le statut "Not Started"
- Le/a créateur/ créatrice
- La date et l'heure de création
- L'auteur de la dernière modification
- La date et l'heure de la dernière modification
- Une priorité non renseignée
- Une criticité non renseignée
- Une date de résolution souhaitée
- Un champ "Détail"
- Un champ "Feature concernée"
- Un champ "Document utile"
- Un champ "Détail résolution"

Cycle de vie d'un ticket

Un ticket possède au total sept statuts qui lui sont attribuable répartis dans trois phases :

PD	In Progress	Complete
Not Started	Assigned → In Progress/ Pending → Reviewing	Done/ Canceled

To-do

La première phase consiste à créer le ticket et à l'expliquer. C'est une phase de définition dans laquelle le ticket sera nommé, la problématique expliquée, puis la criticité et priorité définie.

Une fois tous ces éléments établis, il sera ensuite possible d'assigner le ticket au collaborateur le plus pertinent pour sa résolution.

In Progress

Cette seconde phase intervient une fois que le collaborateur assigné a pris connaissance du ticket et commence à recueillir des informations utiles à sa résolution ainsi qu'à entamer les démarches nécessaires afin de répondre à la demande.

Lors de cette phase, le ticket peut aussi être mis en attente, dans le cas où la personne assignée serait en attente d'informations complémentaires la bloquant dans la résolution.

Une fois la solution apportée le ticket passera en étape de revue où il sera vérifié que sa résolution correspond bien à la demande initiale. Si cette étape n'est pas validée, le ticket pourra retourner à l'état en cours ou en attente selon la situation.

Complete

Une fois l'étape de validation réussie, le ticket sera considéré comme validé et clos.

Il est possible dans certains cas qu'un ticket soit annulé. Cela peut intervenir à n'importe quel moment de son cycle de vie. Il basculera alors dans l'état annulé.

Base de connaissances

Afin de conserver des processus acquis lors de la résolution d'anomalie, et de gagner du temps si des problèmes similaires à ceux déjà rencontrés se produisent, nous avons décidé de mettre en place une base de connaissances.

Cette base sera mise à jour par tous les collaborateurs et ce, de façon normée afin de conserver un document clair et lisible.

Le document est accessible depuis la page de gestion des anomalies mais également via la page ressources dans l'espace Notion du projet.

Il contient, dans un but de suivi de la conformité, un tableau des versions incluant l'auteur de la modification, la date, la version et la description de la dite modification.

Cinq grandes catégories sont initialement proposées dans la base de connaissance (BDD, API, Back Office, Mobile IOs, Mobile Android), ce qui peut évoluer selon les besoins. Au fur à mesure du temps, ces catégories seront remplies avec différentes pages faisant office de fiche label.

Un template de fiche label est imposé afin d'assurer la pertinence dans le contenu de celles-ci. Ces fiches contiennent un tableau des versions identique à celui de la base de connaissances,, un champ problème rencontré et un champ résolution.

| MÉTHODOLOGIE ET STRATÉGIES DE TESTS

Notre approche de test est itérative et incrémentale, alignée sur le cycle de développement du projet. Elle combine différentes stratégies et niveaux de tests pour assurer une couverture complète.

Niveaux de Tests

- Tests unitaires : Réalisés par les développeurs pour valider les services, routers tRPC, schémas Zod et helpers de manière isolée.
- Tests d'intégration: Vérification de l'interaction correcte entre les différents modules avec une base PostgreSQL réelle (pas de mock).
- Tests de régression: Approche snapshot sur les services critiques pour verrouiller les contrats Prisma.
- Tests d'acceptation utilisateur : Réalisés manuellement par les parties prenantes (Product Owner, utilisateurs pilotes) sur l'app mobile (iOS/Android) pour valider que l'application répond aux besoins métier et utilisateurs.

Types de Tests

- Tests Fonctionnels : (automatisés via unitaires + intégration) : Vérification de toutes les fonctionnalités backend décrites dans les spécifications :
 - Mécaniques de jeu (niveaux, progression, objectifs, événements aléatoires, résolution de fin de partie).
 - Système de quiz (questions, réponses, scoring, suivi utilisateur).
 - Gestion de compte utilisateur (inscription, connexion, réinitialisation mot de passe via Better-Auth).
 - Persistance de la progression (levels completion, wallet, holdings, transactions).

- Calculs financiers (intérêts, plus-values price-based et rate-based, portefeuille).
- Tests d'interface utilisateur (UI) et d'expérience utilisateur (UX) : Vérification manuelle de la clarté, de l'intuitivité et de la cohérence de l'interface sur les plateformes cibles (iOS / Android). Pas d'automatisation E2E côté mobile.
- Tests de compatibilité: Validation manuelle du fonctionnement sur les OS/appareils cibles définis (voir section Environnement). Non intégré au CI.
- Tests de performance : Assurés au niveau backend par la stratégie de cache LRU (voir Definition of Done , staticCache 10 min, gameCache 3 s, computeCache 30 s). Pas de test de charge automatisé.
- Tests de régression: Snapshots contractuels sur les services critiques (event, goal, level) rejoués automatiquement à chaque push. Toute modification silencieuse de requête Prisma est détectée immédiatement.

| DOCUMENTATION ET ENVIRONNEMENT

Documentation

- L'équipe QA recevra une formation initiale sur les règles du jeu Cashou, ses objectifs pédagogiques et ses fonctionnalités clés.
- Une formation sur les outils de test et de suivi des anomalies sur Notion sera assurée.
- Une session de présentation pour les nouvelles fonctionnalités impactantes sera organisée avant son déploiement.

Environnements de test

- Développement : Utilisé par les développeurs pour le codage et les tests unitaires.
- Pré-production (Staging) : Environnement miroir de la production, utilisé pour les UAT et les dernières vérifications avant mise en production.
- Production : Environnement live accessible aux utilisateurs finaux.

- Configuration : Les configurations spécifiques (OS, navigateurs, versions, appareils mobiles iOS/Android) sur lesquelles Cashou doit être validé seront listées en annexe et maintenues à jour.

| PHASE DE TESTS

Phase 1 : Tests des fonctionnalités de base (Solo & Quiz)

- Tests unitaires et d'intégration continus.
- Tests système des premiers niveaux de la campagne et du module de quiz.

Phase 2 : Tests multijoueur & Intégration complète

- Tests d'intégration des modules solo, multijoueur et quiz.
- Tests système complets des fonctionnalités multijoueur.
- Tests de régression sur les fonctionnalités de la Phase 1.

Phase 3 : Stabilisation & Préparation

- Tests système complets sur l'ensemble de l'application.
- Tests de compatibilité étendus.
- Tests de performance basiques.
- Campagne intensive de correction de bugs.
- Tests de régression complets.

Phase 4 : Tests d'acceptation utilisateur UAT

- Exécution des scénarios UAT par les utilisateurs clés/Product Owner.

Phase 5 : Préparation lancement & Go/No-Go

- Derniers tests de régression post-UAT.
- Validation finale basée sur les critères d'acceptation.

Gestion des risques et plan de contingences

Méthode AMDEC¹

Un registre des risques disponible dans l'espace notion a été établi via la méthode AMDEC.

Cette méthode consiste à trier des risques selon leur type et à leur attribuer un score de criticité basé sur le calcul suivant :

$$\text{Occurrence} * \text{Défectabilité} * \text{Gravité}$$

Chaque item peut avoir un score allant de 1 (faible) à 4 (critique), la criticité maximale est donc de 64.

Un plan d'action, préventif ou curatif est ensuite établi afin d'essayer d'abaisser la criticité du risque. Afin de ne traiter que les risques les plus pertinents, nous avons décidé d'établir un plan d'action uniquement pour les risques ayant une criticité égale ou supérieure à 24.

Suivi et revu

Des outils de suivi sont mis en place afin de maximiser l'efficacité de notre méthode :

Dans un premier temps, les potentiels risques devront être revus mensuellement, de même pour le plan d'action.

Dans un second temps, des outils de monitoring devront être mis en place afin d'essayer de détecter les différents risques et ce, le plus tôt possible.

Ensuite, un registre des incidents est tenu. Ce registre est disponible dans la page Notion dédiée. Selon la situation, le risque pourra aussi faire l'objet d'un ticket dans l'outil de gestion des anomalies.

Enfin, il est important de mettre en place un plan de communication pertinent, ciblant les bons interlocuteurs lors du déclenchement d'un incident et aussi de sa résolution afin de travailler en transparence mais aussi de se conformer à la législation selon l'événement.

¹ Analyse des Modes de Défaillances, de leurs Effets et de leur Criticité

A noter qu'il est important de garder un suivi statistique du nombre de risque déclenché, de leur temps de résolution et du nombre de résolution afin de pouvoir adapter la méthode de gestion en cas de besoin.

Tous les documents liés à la gestion des risques se situent dans la page homonyme de l'espace Notion.

| Outils de tests

Les outils suivants seront utilisés pour supporter les activités d'assurance qualité :

- Gestion des exigences et user stories : **Notion**
- Gestion des cas de test : **Notion**
- Suivi des anomalies : **Notion**
- Automatisation des tests: **Jest**
- Tests de performance : **Maestro si possible (mais pas de CI)**
- Outil de communication : **Microsoft Teams**

| Conclusion

Ce Plan d'Assurance Qualité établit le cadre nécessaire pour assurer que le serious game "Cashou" soit livré avec un haut niveau de qualité, offrant une expérience utilisateur positive, fiable et pédagogiquement pertinente. Le succès de cette démarche repose sur l'implication de toutes les parties prenantes du projet et sur une communication transparente et continue.

L'équipe QA s'engage à appliquer rigoureusement les processus décrits dans ce document, tout en restant flexible pour s'adapter aux réalités du projet. Ce plan est un document vivant qui sera revu et mis à jour si nécessaire pour refléter l'évolution du projet Cashou. Notre objectif commun est de faire de Cashou un outil d'apprentissage financier de référence, apprécié par ses utilisateurs.